

Kế hoạch v3 chính thức — Auto Pipeline Video AI

Kế hoạch v3 chính thức — Phân shot, Director hiểu Free Fire, thử & so sánh thật ($\leq \$50$)

Chốt 2026-09-23. Bản gốc: `docs/KE_HOACH_V3_CHINH_THUC.md` (bản docx/pdf cùng tên được build từ file này).
Người thực hiện: phiên Claude trên tài khoản phụ — đọc mục "Bàn giao cho phiên thực hiện" ở cuối trước khi bắt đầu.

Context

Bản chạy thử kịch bản Kenta (v2) cho thấy lỗi gốc: **1 cảnh = 1 shot = 1 ảnh = 1 clip** → 56s chỉ có 3 shot (15/15/26s), trong khi phim ngắn chính thức "Kenta's Obsession" 75,7s $\approx$ 43 shot, video kỹ năng Kenta OB55 38,8s $\approx$ 12 shot. Ảnh chỉ là trạng thái đầu của cảnh dài nên nhân vật/vị trí trôi; prompt ảnh và prompt video viết ở hai lúc; Kling chỉ nhận khung đầu. Phân tích ở `docs/PHAN_TICH_2026-09-23_NHAT_QUAN_DIRECTOR_LIPSYNC_STORYBOARD.md`, tóm tắt ở PLAN.md 3.9.

Người dùng chốt (2026-09-23): 1. **Thử cả hai cách rồi so sánh + đánh giá:** gen từng shot riêng vs Kling multi-shot theo nhóm (so thêm với bản v2 hiện tại). 2. **Nghiên cứu nhiều mẫu phong cách Free Fire, nhiều video cho mỗi dạng:** anime CGI, CGI tả thực/kỹ xảo, gameplay trong game, Kelly Show, phim ngắn; thêm **video 3D fan làm nổi tiếng/viral**. 3. **Nhập dựng không cố định** — do kịch bản quyết định. 4. **Ngân sách thử $\leq \$50$, gen 720p hoặc thấp hơn.** 5. Được dùng video kênh chính thức (<https://www.youtube.com/@GarenaFreeFireVN/videos>) làm tư liệu phân tích nội bộ. 6. Chỉ dùng tính năng có API; tính năng chỉ có trên web (Lip Sync, tạo Kling Elements, Voice Design/Clone, Director Workspace, Motion Control, Storyboard Deepix) bỏ qua.

Kết quả mong muốn: Director chia kịch bản thành shot theo phong cách đã học, pipeline chạy theo shot, và một báo cáo so sánh 3 phương án trên cùng kịch bản Kenta bằng video thật ($\leq \$50$) để chốt phương án mặc định.

Cách thực hiện

- Nhánh `v3-shots`; commit + push mỗi giai đoạn; merge `main` sau khi thử thật + sửa lỗi.
- PLAN.md 3.9 đã trở tới file này (chốt 2026-09-23). Khi đổi kế hoạch: sửa file này + PLAN.md, build lại docx/pdf (xem Bàn giao).
- Test viết kèm code; `py -m compileall -q core dashboard` giữa các giai đoạn; chạy toàn bộ test trước khi gen thật (GD6).

- Dữ liệu cũ chạy như cũ: dự án không có `shot_mode` = hành vi v2 (1 cảnh = 1 shot).
- **Mọi lệnh gen tốn tiền: hỏi bạn trước (kèm ước tính)** và luôn nằm dưới trần ngân sách trong code.

GD1 · Thư viện phong cách & ngữ pháp dựng Free Fire (nghiên cứu, không tốn credit gen)

Nguồn: kênh Garena Free Fire VN (+ Official/Global khi cần) và video fan 3D viral (lọc theo lượt xem cao).
Mỗi dạng **≥ 6 video** (tổng ~36–40):

Dạng (<code>style_profile</code>)	Ví dụ đã thấy trên kênh
<code>ANIME_CGI</code>	Kenta's Obsession, Hành Trình Truy Tìm Kho Báu (hoạt hình), Free Fire Daybreak (Kadokawa), Eclipse Rises
<code>REAL_CGI_VFX</code>	Thánh Nữ Tái Sinh – phim kỹ xảo, 9 Years VFX Masterpiece, Cosplay Effects, CGI trailer Bí Ẩn Biển Sâu
<code>INGAME</code>	Kenta Rework OB55, Sleepless with Free Fire, video kỹ năng OB55 nội bộ (<code>D : /2026/OB55</code>)
<code>KELLY_SHOW</code>	Kelly Show OB55 / OB54 / OB53 / Music Festival, Booyah with Kelly
<code>SHORT_FILM</code>	Pitch Party short film, Free Fire x Blue Lock phim ngắn, Free Fire x Gintama CGI short
<code>FAN_3D</code>	video/short 3D fan làm có lượt xem cao nhất (tìm theo lượt xem khi làm)

Cách làm (đã thử được trên “Kenta’s Obsession”): - Trong trình duyệt tích hợp: phát video tắt tiếng → lấy mẫu mỗi 0,2s bằng canvas → phát hiện điểm cắt → bảng khung hình mỗi shot → Claude (trong phiên) gán nhãn từng shot: cỡ cảnh, góc, chuyển động máy, vai trò (hook / thiết lập / hành động / phản ứng / chèn chi tiết / thoại / chuyển / kết), chữ trên hình, giao diện game, hiệu ứng, chuyển cảnh. - Lưu **dữ liệu chữ** (không lưu hình của video vào repo): `research/ff_styles/<STYLE>/<video_id>.json` (danh sách shot + nhãn) và bảng tổng hợp. - Viết `knowledge/ff_styles/<STYLE>.md` cho từng dạng: cấu trúc mở–thân–kết, **phân bố** độ dài shot theo vai trò (khoảng, không phải số cố định), tỉ lệ cỡ cảnh, cách xử lý thoại, chữ/giao diện, hiệu ứng, look (gợi ý World Bible); `knowledge/ff_directing.md` cho phần chung. - **Công cụ lặp lại được** cho file MP4 trên máy: `core/reference_analysis.py` (cắt shot bằng ffmpeg `select='gt(scene,T)',showinfo` theo mẫu `video_analysis.probe` ; khung giữa shot bằng biến thể của `video_analysis.extract_frames` nhận mốc thời gian; ghép bảng khung hình bằng helper ở `core/layout.py:277` để vượt giới hạn 12 ảnh của `llm_runner.MAX_IMAGES` ; gán nhãn qua `llm_runner.ask_json` + prompt mới `prompts/16_reference_shots.md` ; có marker cho `MockLlm`) + khối nhỏ trong ☞ Kho kiến thức.

Đầu ra: `docs/FF_STYLE_RESEARCH.md` (bảng so sánh 6 dạng + nguồn), 6 file phong cách, 1 file chung.

GD2 · Lớp shot + nhịp theo kịch bản

Lưu trữ (tận dụng tối đa pipeline sẵn có): mỗi shot là **một dòng scenes** → jobs, motion prompt, lineage, batch, model router, QC chạy theo shot không cần đổi. - `idx` = số thứ tự shot toàn phim (chỉ **nối thêm**, không đánh số lại vì tên file `videos/{idx}.mp4`, `layouts/S{idx}.png` gắn với `idx`). `data.story_scene`, `data.shot_no`, `data.sequence` (= nhóm liên mạch). - Bảng mới `story_scenes(project_id, idx, heading, text, data)` giữ kịch bản gốc do `script_parser.import_scenes` ghi; shot **không chép** `text` của cảnh (tránh lặp thoại và làm “Δ cũ” hàng loạt) — `text` của shot = mô tả shot, `dialogue` = đúng các câu của shot; `dialogue.scene_lines` coi `[]` là “không có thoại” với dòng có `shot_no` (sửa cả `update_scene`, `_empty` trong `store_scene_analysis`). - Cột `projects.shot_mode` (NULL = v2, `per_shot`, `multishot`), `projects.style_profile` (6 dạng ở GD1; khác `genre`).

Director v3 (`prompts/01...` + validator `llm_io`): mỗi cảnh trả `shots[]: size` (ECU/CU/MCU/MS/WS/EWS/GAME_TPS), `angle`, `camera_move`, `role`, `duration_s` (0,8–15), `start_frame`, `end_state` (nếu đổi trạng thái), `action` (1 động từ chính), `characters`, `dialogue` (câu thoại của shot), `continuous_with_next`, `refs` (tài nguyên + vai trò @Hình). `store_shot_plan()` mới: thay các dòng cảnh chưa có job bằng các dòng shot (từ chối nếu đã có job, trừ khi bạn xác nhận “làm lại”). `build_director_bundle` đọc `story_scenes` và nạp `knowledge/ff_styles/<style>.md` cảnh `genre_text` (`core/prompts.py:89`).

Nhịp do kịch bản quyết định (không có con số cố định): Director chọn độ dài theo nhịp của từng beat + phân bố của phong cách; kiểm tra chỉ chặn điều sai rõ: shot có thoại ngắn hơn thời gian nói (ước lượng, sau TTS dùng giọng thật qua `voice.fit_durations`), vượt tối đa của model, cảnh thoại dài không có shot đáp/phản ứng; cảnh báo mềm khi cỡ cảnh lặp nhiều lần liên.

Độ dài tối thiểu của model (Kling 3s, Seedance 4s — `clipai.effective_duration`): lưu `data.use_s` (giây dùng) + `trim_start`; `final_cut.clip_seconds / render / voice.place_on_timeline / subtitles.build_cues` cùng đọc một hàm độ dài đã cắt; `model_router.plan` + `cost` tính theo giây bị tính tiền thật; thêm `min_sec` vào `data/video_models.json`; thống nhất giới hạn thời lượng (validator, UI step1, prompt 01).

Giao diện: helper nhấn `s02.3` (thay “Cảnh {idx}” ở step1–5, `common.scene_expander`); Bước 1 thành **bảng shot nhóm theo cảnh** (sửa từng shot, thêm shot vào cảnh), chọn `style_profile` + `shot_mode` ở  Định dạng. Autopilot: bước “phân shot” sau Director, `MAX_SCENES` đếm cảnh kịch bản, tràn job theo số shot.

GD3 · Nhất quán ảnh ↔ prompt ↔ video + Kling multi-shot

- **Một nguồn cho mọi prompt:** prompt ảnh (khung đầu) và prompt video cùng dựng từ bản ghi shot; một bảng nhãn tài nguyên dùng chung cho Deepix (`assets.reference_note`) và Seedance.
- **Khung đầu + khung cuối (Seedance, có API):** shot `continuous_with_next` → `last_frame` = ảnh khung đầu đã duyệt của shot kế tiếp (không tốn ảnh thêm). Thêm `role: last_frame` trong `ClipAIVideoProvider.submit`.
- **Nối liền trong nhóm:** ảnh shot sau gen với ảnh shot trước làm tham chiếu (`runner.chain_previous`), gen **tuần tự trong một nhóm** (shot sau chờ ảnh shot trước được duyệt) thay vì gửi cả lô.
- **Một model cho một nhóm:** `model_router` chọn theo nhóm (`sequence`); shot cận/trung nhân vật chính ưu tiên Seedance (có ảnh nhân vật).

- **shot_mode = multishot**: mỗi nhóm = 1 job Kling multi-shot (`multi_prompt` , tổng ≤ 15s, mỗi shot ≥ 3s) gắn vào shot đầu nhóm (job “trưởng nhóm”, cột `jobs.group_leader`); các shot còn lại có job “đi kèm” không tự gửi. Khi tải về: ffmpeg cắt theo độ dài từng shot → `videos/{idx}.mp4` cho từng shot, đánh dấu các job đi kèm xong → QC/lineage/giọng/phụ đề/render vẫn theo shot. Thay cho `core/experiments.py` (giữ lại làm tham khảo).
- **QC đồng bộ cả bộ clip**: mở rộng `claude_tasks` set-consistency sang khung giữa của mọi clip + điểm nối giữa 2 shot liền nhau.

GD4 · Giọng Việt (chỉ API) + storyboard theo shot

- Chỉ đề xuất giọng có `vi` (5 giọng: 4 nam, 1 nữ); cảnh báo khi nhân vật nữ nhiều hơn giọng nữ; `prompts/15_voice_casting.md` ưu tiên `vi`.
- `data/pronunciation_vi.json` (loot, skill, Booyah, tên nhân vật...) áp vào chữ gửi TTS (phụ đề giữ nguyên); nút “nghe thử câu mẫu tiếng Việt”.
- So `eleven_v3` vs `eleven_multilingual_v2` trên 2 câu × 5 giọng (tính trong ngân sách).
- **Storyboard theo shot**: Bước 2 hiện lưới ảnh khung đầu theo thứ tự shot (duyet nhịp trước khi gen video); animatic theo shot (đã có `delivery.animatic` , chạy theo dòng).

GD5 · Trần ngân sách, chế độ thử rẻ, so sánh

- **Trần ngân sách cứng** `BUDGET_USD` (mặc định 50, trong ☼): tổng chi (video + âm thanh theo `usage_events` , mọi dự án, từ mốc bắt đầu thử) + ước tính lệnh sắp gửi > trần → chặn trước khi gửi (trong `_Runner.submit_pending` , audio submit, autopilot) và báo. Ảnh Deepix chưa có giá → đếm số ảnh, trần 80 ảnh cho đợt thử.
- **Chế độ thử rẻ** (cờ dự án `test_quality`): Kling `std` , Seedance 2.0 Fast 720p (hoặc 480p), không cho lên 1080p (chặn `resolution` trong `runner._submit_kwargs`).
- **Nhân bản dự án** (kịch bản, Character Bible, Lock, giọng, tài nguyên, World Bible) để có 3 dự án cùng xuất phát.
- **Bảng so sánh** trong 📊 Theo dõi: 3 video cạnh nhau + chỉ số: số shot, độ dài shot trung bình/phân bố, chi phí thật, thời gian gen, số lần gen lại, điểm QC ảnh/video, điểm đồng bộ bộ clip, điểm nối shot, thoại vừa clip; ô chấm điểm của bạn (1–5): nhân vật nhất quán, bối cảnh nhất quán, nhịp, “chất Free Fire”, tổng thể.

GD6 · Thử thật & đánh giá (≤ \$50, 720p)

Cùng kịch bản Kenta, cùng Character Bible/giọng, phong cách chọn theo kết quả GD1 (dự kiến `KELLY_SHOW` hoặc `ANIME_CGI` — hỏi bạn lúc đó):

Phương án	Cách gen	Ước tính video
V0 — bản v2 hiện tại	3 cảnh = 3 clip	~56s × ~\$0,10 ≈ \$6

Phương án	Cách gen	Ước tính video
V1 — từng shot	~12–18 shot, ảnh riêng từng shot, Seedance Fast/Kling std, khung đầu+cuối	~65s × ~\$0,11 ≈ \$7–8
V2 — Kling multi-shot	3–5 nhóm × 1 lệnh multi-shot	~56s × \$0,08 ≈ \$5

Cộng dự phòng gen lại ×1,5 + TTS → **ước tính ~\$25–30, trần cứng \$50**. Hỏi bạn trước lệnh gen tốn tiền đầu tiên. Sau khi có video: chạy QC/chỉ số tự động, bạn chấm điểm → docs/V3_AB_REPORT.md (bảng chỉ số, ưu/nhược, chi phí/giây thành phẩm, đề xuất phương án mặc định và khi nào dùng phương án kia) → cập nhật PLAN.md (quyết định mặc định), TODO.md, build docs, merge main.

File chính

- Mới: core/reference_analysis.py, core/shots.py (store_shot_plan, nhãn, độ dài dùng), core/budget.py, prompts/16_reference_shots.md, knowledge/ff_styles/*.md, knowledge/ff_directing.md, research/ff_styles/**/*.json, data/pronunciation_vi.json, docs/FF_STYLE_RESEARCH.md, docs/V3_AB_REPORT.md, tests/test_v3.py.
- Sửa: core/db.py (story_scenes, shot_mode, style_profile, test_quality, jobs.group_leader), core/script_parser.py, core/llm_io.py, core/prompts.py, prompts/01..., core/dialogue.py, core/voice.py, core/subtitles.py, core/final_cut.py, core/runner.py, core/adapters/clipai.py, core/model_router.py, data/video_models.json, core/cost.py, core/claude_tasks.py, core/autopilot.py, dashboard/steps/step1-5.py, dashboard/common.py, dashboard/admin.py.
- Dùng lại: video_analysis.probe/extract_frames, layout ghép bảng ảnh, llm_runner.ask_json, runner.chain_previous/previous_frame_job, voice.fit_durations/place_on_timeline, delivery.deliver/animatic, cost.spend_summary/record_usage, autopilot._daily_cap (mẫu cho trần ngân sách), experiments.kling_multishot (mẫu request multi-shot).

Kiểm chứng

- Unit test (tests/test_v3.py + bộ cũ): tách shot từ JSON Director v3, shot không lặp thoại, “⚠ cũ” chỉ shot bị sửa, độ dài dùng/cắt khớp giữa render–giọng–phụ đề, multi-shot trưởng nhóm cắt đúng thành clip từng shot, Seedance nhận last_frame, trần ngân sách chặn đúng, chế độ thử rẻ không lên 1080p, dự án v2 cũ chạy như trước.
- Chạy giả lập qua trình duyệt (MOCK_REAL_MEDIA=1) kịch bản Kenta ở cả 3 phương án tới bản giao.
- GD6 thật: 3 video + báo cáo so sánh + điểm của bạn; tổng chi thật ≤ \$50 (kiểm trong sổ chi).

Bàn giao cho phiên thực hiện (tài khoản phụ)

Bắt đầu

1. `git pull` trên `main` (commit chốt kế hoạch này), tạo nhánh `v3-shots`.
2. Đọc theo thứ tự: `CLAUDE.md` (quy ước: luôn commit + push; sửa `PLAN.md` thì chạy `bash tools/build_docs.sh`; luôn cập nhật `TODO.md` trong cùng commit), file này, `PLAN.md` 3.8–3.9 + Mục 5, `TODO.md`, `docs/V2_TEST_REPORT.md`, `docs/PHAN_TICH_2026-09-23_NHAT_QUAN_DIRECTOR_LIPSYNC_STORYBOARD.md`.
3. Giao tiếp với người dùng **bằng tiếng Việt**. Hỏi trước mọi việc tốn credit.

Trạng thái hiện tại

- Dashboard v2 đã xong và ở `main` (632 test pass): lineage “`Δ` cũ”, Director JSON v2 + khóa , tỉ lệ khung theo dự án (mặc định 9:16), model theo từng cảnh (`core/model_router.py`, `data/video_models.json`, 3 mức ưu tiên), giọng TTS theo nhân vật (`core/voice.py`), bản giao dựng → phụ đề → card cuối → kích thước (`core/delivery.py`), QC video, chế độ tự động v2.
- Chạy thử không tốn credit: `py tools/seed_demo.py` rồi `powershell -File tools/run_demo.ps1` (đã bật `MOCK_REAL_MEDIA=1`: ảnh/clip giả lập là file thật nhỏ, dựng/xuất được). Bộ test: `py -m unittest discover -s tests -t .` (~5 phút).

Môi trường (Windows)

- Python gọi bằng `py` (trong Git Bash không có `python`). `ffmpeg` có sẵn trong `PATH`. `pandoc` + `Chrome` cho build docs.
- Token `CLIPAI_TOKEN`, `DEEPIX_TOKEN` nằm trong biến môi trường người dùng — **không in, không ghi vào repo/log**. Claude cho Dashboard: `LLM_PROVIDER=claude_cli` (dùng chung hạn mức tài khoản).
- Kịch bản thử: “KỊCH BẢN FREE FIRE: KENTA XUYỀN TƯỜNG CƯỚP KILL?!” (3 cảnh, Kelly/Maxim/Kenta, 14 câu thoại, “TEXT CUỐI”). Nguyên văn ở `samples/kenta_xuyen_tuong_cuop_kill.txt`.
- Video Free Fire nội bộ trên máy: `D:/2026/0B55/` (combo Kenta, tình huống Kenta...), `D:/2025/` — dùng để phân tích, không đưa vào repo.

Sự thật về API (tra 2026-09-23, chỉ đọc)

- Token dùng được 12 endpoint trong skill `clipai 1.3.1` (Get this Skill to Claude/clipai-1.3.1/clipai/reference.md): Kling Omni (có `multi_shot` / `multi_prompt`, `image_list` chỉ `first_frame`, `element_list` — nhưng tạo Element chỉ trên web), Seedance (content với role `first_frame` / `last_frame` / `reference_image` / `reference_video` / `reference_audio`; 2.0 dài 4–15s, 2.5 dài 4–30s; 480p/720p), video-list, âm thanh (TTS/SFX/nhạc, voice-actors), kho chủ thể Seedance.
- Endpoint chỉ có trên web (token trả code 1102 LoginErr): `/kling/lip-sync`, `/kling/element-create|list`, `/kling/custom-voice-list`, `/sound/voice-actors/design|clone`, `/kling/motion-control`, Director Workspace → **không dùng** (quyết định 6).
- Giọng có `vi` (5 giọng, id trùng 2 bản): Xinghe Jiang (30002/2, nam), Austin (30003/3, nam), **Arabella (30007/7, nữ)**, Ethan Zhang (30017/17, nam), Mark (30020/20, nam). Không có giọng gốc Việt. Truyền `language_code="vi"` cho TTS gây HTTP 400 → không truyền.
- Độ dài tối thiểu: Kling 3s, Seedance 4s (`core/adapters/clipai.py::effective_duration`). Giá ước tính (USD/giây): Kling 0,08; Seedance 2.0 0,15; 2.0 Fast 0,12; 2.5 0,23 (`data/pricing.json`, chưa đo thật). Deepix chưa có giá.

Nghiên cứu video YouTube (GD1)

- **Trình duyệt tích hợp mở được YouTube**; tải bằng dòng lệnh (urllib) bị 404, chỉ RSS đọc được. Kênh: <https://www.youtube.com/@GarenaFreeFireVN/videos> (tab Popular để lấy video nhiều lượt xem).
- Cách đã chạy được trên "Kenta's Obsession" (<https://www.youtube.com/watch?v=cUQ1PhwvfAE>): mở trang video, rồi chạy JS (từng đoạn ≤ 20 giây video mỗi lần gọi vì giới hạn 45 giây của công cụ):

```
window.__scan = async function(from, to, step){
  const v=document.querySelector('video'); v.muted=true; v.pause();
  const c=document.createElement('canvas'); c.width=48; c.height=27;
  const x=c.getContext('2d',{willReadFrequently:true});
  window.__diffs=window.__diffs||[]; let prev=window.__prev||null;
  for(let t=from;t<to;t+=step){
    await new Promise(r=>{const h=()=>{v.removeEventListener('seeked',h);r()};
      v.addEventListener('seeked',h); v.currentTime=t; setTimeout(r,800)});
    x.drawImage(v,0,0,48,27); const d=x.getImageData(0,0,48,27).data; const g=new
      Float32Array(48*27);
    for(let i=0;i<g.length;i++) g[i]=(d[i*4]+d[i*4+1]+d[i*4+2])/3;
    if(prev){let s=0; for(let i=0;i<g.length;i++) s+=Math.abs(g[i]-prev[i]);
      window.__diffs.push([+t.toFixed(2), +(s/g.length).toFixed(1)]);}
    prev=g;
  }
  window.__prev=prev; return window.__diffs.length;
};
// gọi: window.__diffs=[]; window.__prev=null; await __scan(0,20,0.2); await __scan(20,40,0.2); ...
// điểm cắt: diff > max(18, 4 * trung vị), bỏ các điểm liền kề
```

- Kết quả mẫu: Kenta's Obsession 75,7s, ~42 điểm cắt (1,6 · 4 · 6,2 · 7,4 · 9,8 · 11,2 · 11,6 · 13,6 · 14,2 · 15,6 · 18,4 · 21,2 · 22,2 · 24 · 24,6 · 25,4 · 25,8 · 26,2 · 27,6 · 28,2 · 29,2 · 30,4 · 31 · 32,2 · 34,8 · 35,8 · 37,8 · 39,8 · 40,6 · 41,6 · 42,6 · 43,6 · 44,8 · 46,8 · 51,8 · 56 · 60,2 · 61,4 · 64 · 66 · 70,2 · 73,6s); phong cách anime, xen toàn–trung–cận–cận đặc tả–chèn hiệu ứng, logo cuối. Video nội bộ Kenta OB55 (ffmpeg select='gt(scene,0.30)'): 38,8s, ~12 shot, mở bằng nhân vật tạo dáng + tiêu đề, thân là gameplay góc thứ ba sau lưng + chữ chương, kết bằng câu kêu gọi.
- Xem khung hình: vẽ khung giữa mỗi shot vào một canvas lớn, gán thành `<img>` phủ lên trang rồi chụp màn hình (kết quả `toDataURL` quá lớn để trả về trực tiếp; chia bảng thành từng phần nếu cần xem rõ).
- Chỉ lưu dữ liệu chữ (mốc cắt, nhãn, thống kê) vào repo; không lưu hình của video.

Bài học kỹ thuật (tránh lặp lỗi)

- Heredoc bash chứa tiếng Việt / ký tự thoát dễ hỏng file → dùng Write/Edit tool hoặc script Python trong scratchpad.
- Streamlit không tự nạp lại module `core/`, `dashboard/steps/` khi sửa → khởi động lại server để thử.
- Widget Streamlit có `key` giữ giá trị cũ khi dữ liệu đổi ở nơi khác → key phải gắn theo giá trị trong dữ liệu (đã sửa ở bảng trộn âm Bước 5) hoặc xóa state sau khi đổi.

- Thanh bước là `st.radio` có nhãn động → phải gán lại `st.session_state["step"]` trước khi vẽ (đã sửa trong `dashboard/app.py`).
- Callback nút (`on_click`) chạy ở luồng khác → mở kết nối SQLite mới trong callback.
- `$` trong `st.caption/markdown` bị hiểu là công thức → viết `\\$`.